Les bases de données relationnelles et Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Les bases de données **3**

A. Qu'est-ce qu'une base de données ? 3

B. Les types de relations 5

C. Exercice : Quiz 6

III. Manipuler les bases de données SQL **7**

A. Les opérations 7

B. Quelques exemples 8

C. Les ORMs 10

D. Exercice : Quiz 10

IV. Essentiel **11**

V. Auto-évaluation **11**

A. Exercice 11

B. Test 12

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : Bases en Python

Contexte

Tout programmeur se trouve à un moment donné confronté à la gestion d'une base de données, peu importe le type d'application qu'il est amené à développer.

Les bases de données peuvent être de différents types, et il existe plusieurs architectures de bases de données.

L'objectif de ce cours est de donner les éléments clés pour comprendre comment fonctionne une base de données, comment elle peut être organisée, mais aussi d'apprendre comment on peut manipuler une base de données à l'aide de Python. Nous verrons quelques exemples qui vous permettront d'y voir plus clair.

II. Les bases de données

A. Qu'est-ce qu'une base de données ?

Supposons que vous êtes en charge de développer le site web d'une bibliothèque. Ce site contient une section à travers laquelle chaque utilisateur est capable de visualiser la liste des ouvrages disponibles, mais aussi de les louer, lorsqu'ils sont disponibles.

Une question qui vous viendra rapidement à l'esprit est : comment stocker les informations afin qu'elles puissent facilement être chargées et modifiées à travers une page web ?

La réponse est : grâce à l'utilisation d'une base de données. Celle-ci vous permettra en effet de recenser l'ensemble des ouvrages, de pouvoir leur accrocher un statut de location (disponible ou non disponible), mais aussi de pouvoir éventuellement ajouter des livres (si vous êtes le propriétaire de la bibliothèque, par exemple).

Par exemple, votre base de données pourrait contenir les informations suivantes :

Titre	Auteur	Disponibilité
Livre A	Auteur 1	Oui
...	...	...

Relationnelles ou non-relationnelles ?

Les bases de données peuvent être de deux types : relationnelles ou … non-relationnelles !

Les premières sont généralement appelées bases de données **SQL** (« *Structured Query Language* »), tandis que les secondes sont généralement appelées **NoSQL**.

Une base de données relationnelle est constituée de tables. Chacune de ces tables est elle-même constituée de lignes et de colonnes. Les lignes sont des enregistrements, tandis que les colonnes sont les différents attributs qui caractérisent ces enregistrements.

Toutes ces tables sont liées entre elles, ces liens sont établis à l'aide de 'clés' : clés primaires, et clés étrangères (Primary Key, Foreign Key). L'appellation 'base de données SQL' vient du fait que l'interaction avec ce type de bases de données se fait à l'aide d'un langage spécifique, appelé « *Structured Query Language* ». Les bases de données SQL les plus connues sont MySQL, SQLite, MariaDB, etc.

Dans le cas d'une **base de données non relationnelle**, les données ne sont pas structurées sous formes de tables. La structure est beaucoup plus « *libre* », puisque les différents enregistrements peuvent être des fichiers, des graphes, des documents, etc. Les bases de données NoSQL les plus connues sont MongoDB, Cassandra, Redis, etc.

Le choix de l'utilisation du type de base de données dépendra généralement des contraintes applicables, notamment la scalabilité (capacité à faire évoluer la structure des données), la croissance de la taille de la base de données, le format des enregistrements, etc.

Dans le cadre de ce cours, nous allons nous intéresser particulièrement aux bases de données de type relationnelles (SQL). Voyons comment étendre notre exemple de gestion de bibliothèque pour cerner concrètement la définition d'une base de données relationnelle.

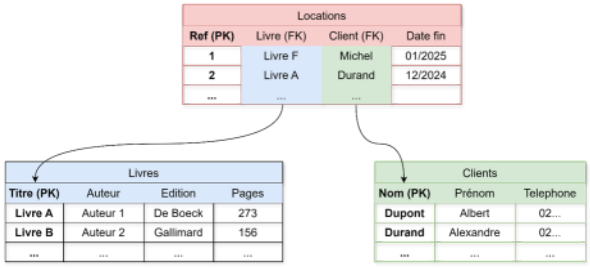
Remarque

Une base de données non-relationnelle également appelé « *NoSQL* » pour « *Not only SQL database* ». Ceci vient du fait que dans certains cas particuliers, en fonction de la nature des données qui sont stockées dans une base NoSQL, il est possible de réaliser des transactions à l'aide du langage SQL, comme on le ferait pour une base de données relationnelle. Pour plus de facilité, dans le reste de ce cours, lorsque nous ferons référence à une base de données sans préciser son type, il s'agira implicitement d'une base de données relationnelles (SQL).

Reprenons donc le cas de notre bibliothèque. Nous souhaitons stocker toutes ces informations dans une base de données (relationnelle). Une façon de le faire serait de définir 3 tables :

- Les livres
- Les clients
- Les locations en cours

Cela pourrait par exemple ressembler à ceci :



Dans cet exemple, nous avons deux tables de base : la table des '**Livres'** (en bleu), dans laquelle on répertorie tout simplement les informations sur les livres disponibles en bibliothèque, ainsi qu'une table de **'Clients'** (en vert), dans laquelle on répertorie les clients inscrits à la bibliothèque.

La table '**Locations**' (en rouge) établi des relations entre les tables élémentaires : c'est la notion de base de données relationnelle.

Pour chacune de ces tables, la 1ère colonne (1er attribut de chaque table) est un identifiant unique qui permet de pointer de manière non-univoque chaque ligne (enregistrement) de la table. C'est ce qu'on appelle la clé primaire (« *Primary Key* » = « *PK* »).

Dans la table 'Locations', des liens sont établis vers les enregistrements de la table 'Livres' et de la table 'Clients'. Ces liens sont appelés clés étrangères (« *Foreign Key* » = « *FK* »), tout simplement car ce sont des liens vers des tables « *étrangères* ».

Avec l'exemple précédent, la notion de base de données relationnelle devrait vous paraître plus claire. Dans la section suivante, nous allons voir qu'il existe plusieurs types de relations qui caractérisent les bases de données relationnelles.

Tous droits réservés à STUDI - Reproduction interdite

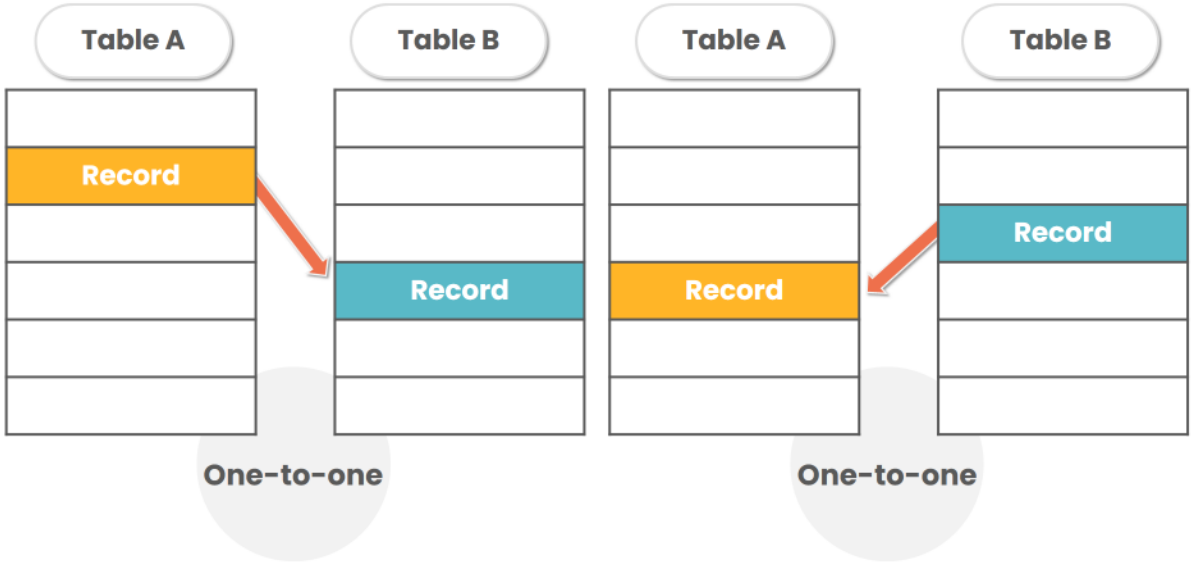
B. Les types de relations

Les relations, comme nous l'avons vu dans l'exemple, permettent de faire des liens entre les différents enregistrements des différentes tables.

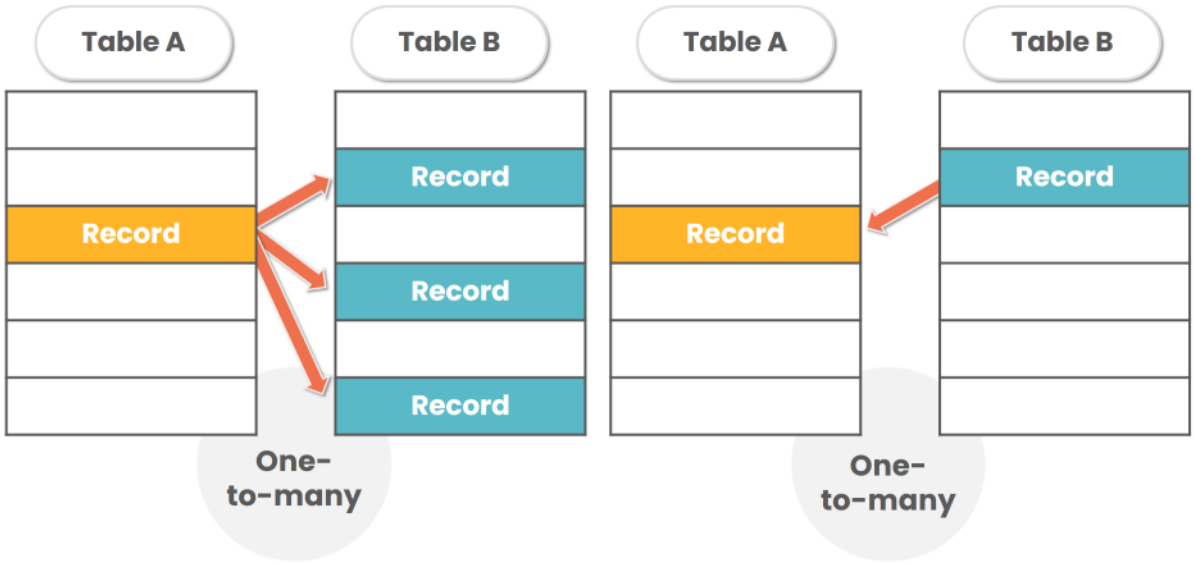
Cependant, il existe plusieurs types de relations :

- **One-to-one** : un enregistrement d'une table est lié à un seul enregistrement d'une autre table.
- **One-to-many** : un enregistrement d'une table est lié à plusieurs enregistrements d'une autre table.
- **Many-to-many** : un enregistrement d'une table peut être lié à plusieurs enregistrements d'une autre table, et inversement, plusieurs enregistrement de la première table peuvent être liés à un enregistrement de la seconde.

Toujours dans le cadre de notre exemple, on pourrait typiquement avoir une relation one-to many. En effet, un client peut louer plusieurs livres différents, cependant, un livre ne peut être loué que par un client à la fois.

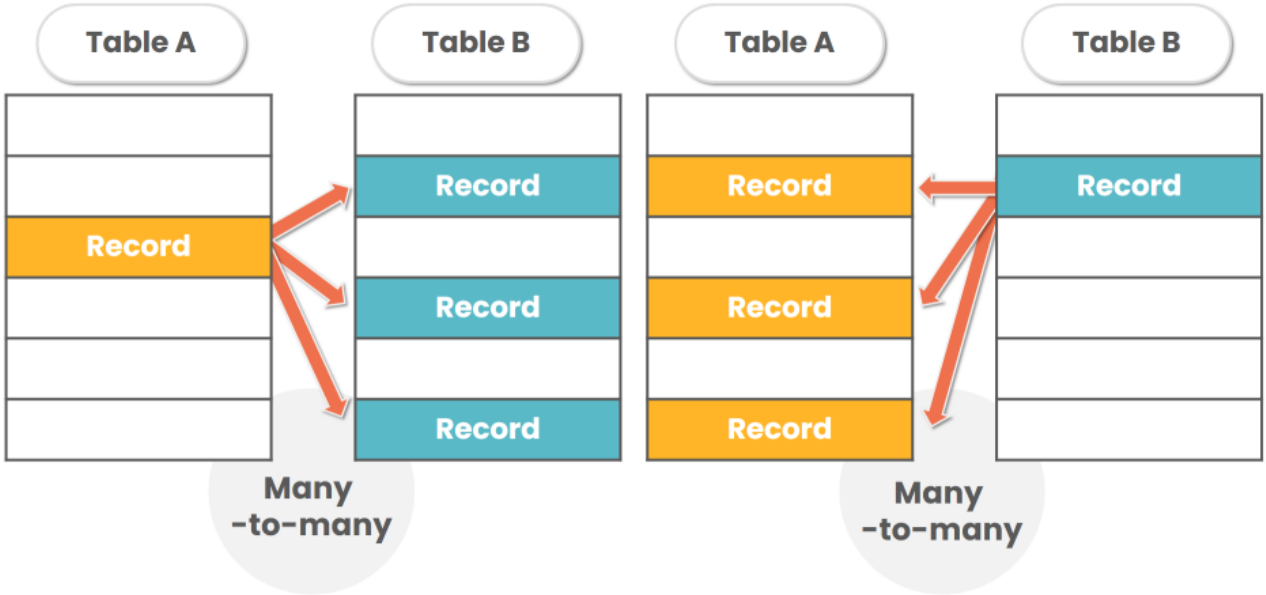


Relation de type « *one-to-one* »



Tous droits réservés à STUDI - Reproduction interdite

Relation de type « *one-to-many* »



Relation de type « *many-to-many* »

C. Exercice : Quiz

[solution n°1 p.13]

Question 1

Une base de données, c'est…

- Un programme qui gère le stockage des données
- Une solution complexe qu'on met en œuvre uniquement pour des gros projets
- Un moyen simple et efficace de stocker des données

Question 2

L'acronyme NoSQL signifie qu'une base de données non-relationnelle ne peut jamais être manipulée à l'aide de requêtes SQL.

- Vrai
- Faux

Question 3

Une base de données relationnelle peut utiliser des fichiers ou des graphes comme enregistrements.

- Vrai
- Faux

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Qu'est ce qu'une clé primaire ?

- Un enregistrement dans une table
- Un identifiant unique dans une table
- Un type de relation dans une base de données non-relationnelle

Question 5

Combien de types de relations existe-t-il dans les bases de données relationnelles ?

- 1
- 2
- 3

III. Manipuler les bases de données SQL

A. Les opérations

Il existe plusieurs opérations qui permettent…

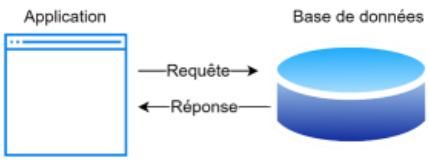
- de **lire**
- de **modifier**
- de **supprimer**
- d'**ajouter**

…des éléments dans une base de données. Cependant, sur la dizaine d'opérations autorisées, seules 4 sont généralement utilisées :

- **C**REATE : ajoute un nouvel enregistrement,
- **R**EAD : récupère le contenu d'un ou plusieurs enregistrements,
- **U**PDATE : met à jour un enregistrement existant,
- **D**ELETE : supprime un enregistrement.

L'ensemble des premières lettres de chaque action forme le mot clé « **CRUD** », que vous verrez souvent dans la littérature. Ces 4 actions sont finalement regroupées sous cet acronyme.

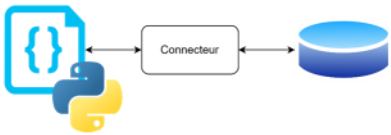
Pour interagir avec une base de données, il faut lui envoyer une requête, formulée sous forme d'instruction SQL. Cette requête est interprétée par la base de données, qui renvoie une réponse contenant les informations demandées, ou bien une confirmation de modification le cas échéant. Il s'agit donc d'un protocole requête-réponse.



Tous droits réservés à STUDI - Reproduction interdite

B. Quelques exemples

Afin d'interagir avec une base de données depuis l'environnement Python, il est nécessaire d'installer ce qu'on appelle un connecteur. Un connecteur est tout simplement une interface, qui vous permettra de vous connecter à votre base de données et de lui envoyer des requêtes SQL depuis votre application Python.



Dans ce cours, nous utiliserons la base de données SQLite. Il s'agit d'une base de données relationnelle extrêmement réputée, qui a l'avantage d'être très légère et qui peut facilement être utilisée pour créer des bases de données stockées sur une machine physique.

Par ailleurs, Python comporte nativement un connecteur vers une base de données SQLite. Il n'y a donc pas besoin d'installer de module complémentaire !

Commençons avec un exemple simple dans lequel nous allons créer notre base de données.

Exemple **Créer une base de données SQLite**

Commençons par importer le module sqlite3 (préinstallé avec Python), et créons ensuite la base de données qu'on appellera 'database.db' :

```
1 import sqlite3
2
3 myDB = sqlite3.connect("database.db")
```

Une fois ce code exécuté, vous verrez apparaitre dans le répertoire de travail, un fichier appelé 'database.db'. Il s'agit donc de la base de données que nous venons de créer grâce à la fonction 'connect()'.

Notez que si vous utilisez cette fonction sur une base de donnée existante, la fonction connect() permet alors tout simplement de se connecter à la base de données spécifiée.

Exemple

Une fois notre base de données créée, nous allons créer une table dans celle-ci, et y insérer... des données. Supposons qu'on souhaite créer une table d'utilisateurs. Cette table contiendra les attributs suivants :

- Pseudo
- Adresse email
- Prénom

Avant de passer au code à proprement parler, précisons deux éléments :

- Pour adresser des requêtes à notre base de données, il est nécessaire de passer par un objet 'Cursor'. Cet objet permet de spécifier les requêtes et d'interfacer directement avec notre base de données. Cet objet est défini dans la librairie sqlite3.
- Nous verrons que les requêtes sont formatées en langage SQL. Si vous ne connaissez pas ce langage, pas de panique : c'est un langage descriptif, et vous verrez qu'il est très simple à apprendre.

```
1 cur = myDB.cursor() # Notre objet Cursor
2
3 cur.execute("CREATE TABLE utilisateurs(pseudo, email, prenom)")
4
5 cur.execute("""
6     INSERT INTO utilisateurs VALUES
7         ('pseudo-user1', 'user1@domain.com', 'Jean'),
```


Tous droits réservés à STUDI - Reproduction interdite

```
        ('pseudo-user2', 'user2@domain.com', 'David')
    """)

myDB.commit()
```

Dans ce code, nous créons notre objet 'Cursor'. Ensuite, nous créons une table 'utilisateurs' avec 3 colonnes, et y insérons deux lignes (deux enregistrements).

Attention, afin de réellement appliquer notre requête vers notre base de données, il est nécessaire de faire appel à la fonction 'commit()' de notre objet myDB. Si vous ne le faites pas, votre requête ne sera pas prise en compte par votre base de données.

Exemple **Récupérer des éléments dans la base de données**

Une fois qu'on a enregistré des éléments dans une base de données, on souhaite pouvoir récupérer les informations stockées.

Supposons qu'on souhaite récupérer, dans notre table 'utilisateurs', les informations relatives à l'utilisateur dont le pseudo est 'pseudo-user2' :

```
res = cur.execute("SELECT email, prenom FROM utilisateurs WHERE pseudo='pseudo-user2'")
print(res.fetchall())
```

Comme vous le voyez, la requête est extrêmement simple. SELECT permet de spécifier les attributs à récupérer, FROM permet de spécifier la table qu'on interroge, et WHERE permet de filtrer la requête. Ici, on souhaite filtrer sur la valeur de l'attribut 'pseudo' en spécifiant sa valeur : 'pseudo-user2'.

Ceci devrait vous donner le résultat suivant :

```
> [('user2@domain.com', 'David')]
```

C'est donc bien l'adresse mail et le prénom de notre utilisateur au pseudo 'pseudo-user2'.

Exemple

Dans certains cas, vous serez amenés à exécuter des opérations sur plusieurs enregistrements d'une table. Pour cela, il est possible d'utiliser l'instruction *for…in…* de Python, comme le montre l'exemple suivant :

```
for row in cur.execute("SELECT email, prenom FROM utilisateurs"):
    print(row)
```

Ce qui donnera le résultat suivant :

```
> ('user1@domain.com', 'Jean')
> ('user2@domain.com', 'David')
```

On obtient donc l'email et le prénom de l'ensemble des enregistrements.

Exemple **Création d'une table avec 'Foreign keys'**

Parfois, il est nécessaire de lier plusieurs tables entre-elles. C'est le concept que nous avons vu dans la première partie du cours, qui était illustré par notre base de données relative à une bibliothèque.

Pour ce faire, il suffit, lors de la création de la table, de spécifier que certaines colonnes doivent être liées à des attributs dans d'autres tables.

Tous droits réservés à STUDI - Reproduction interdite

Supposons que nous disposions de 3 tables : maTable, maTable2 et maTable3. Nous souhaitons que les attributs 'Attribut2' et 'Attribut3' soient liés aux colonnes 'Attribut1' des tables maTable2 et maTable3 :

```
cur.execute("""CREATE TABLE maTable(Attribut1,
            Attribut2,
            Attribut3,
            Attribut4,
            FOREIGN KEY(Attribut2) REFERENCES maTable2(Attribut1),
            FOREIGN KEY(Attribut3) REFERENCES maTable3(Attribut1))""")
```

Dans ce cas, il existera donc des liens entre les 3 tables dont il est question.

C. Les ORMs

Dans les quelques exemples précédents, nous avons vu comment interagir avec une base de données relationnelle à l'aide du langage SQL, à travers un script Python.

Vous l'aurez remarqué, ce type d'interaction n'est pas toujours évident :

- Il nécessite de connaitre les commandes SQL pour construire les requêtes.
- L'intégration du SQL forme un ensemble hétérogène avec le reste du code Python.

Pour supprimer ces deux inconvénients, il est possible d'utiliser des ORMs. De manière très simple, un ORM (Object-Relational mapper) est une façon de lier une base de données à la notion d'objets (les mêmes objets que dans la programmation orientée objet), un des principe fondamentaux de la programmation en Python.

Avec les ORMs, le fait de manipuler une base de donnée devient totalement transparent, grâce au niveau d'abstraction apporté par ces ORMs.

Dans un prochain cours, nous aborderons en détails la notion d'ORM, et vous verrez que manipuler une base de données deviendra encore plus simple !

D. Exercice : Quiz

[solution n°2 p.14]

Question 1

CRUD est l'acronyme qui désigne …

- ○ Un ensemble d'opérations utiles pour manipuler une base de données
- ○ Un type de base de données
- ○ Un ensemble de relations qui peuvent caractériser une base de données

Question 2

L'interaction avec une base de données fonctionne sur base d'un principe requête-réponse.

- ○ Vrai
- ○ Faux

Question 3

SQLite est une base de données relationnelle.

- ○ Vrai
- ○ Faux

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Que ne faut-il pas oublier lorsqu'on souhaite effectuer une modification dans une base de données ?

- Il faut commiter les modifications à l'aide de la fonction 'commit()'
- Il faut enregistrer son script Python
- Il faut clôturer la connexion à la base de données

Question 5

ORM est l'acronyme de … :

- One Relation Mapping
- Object Relational Mapper
- Object Regression Mapper

IV. Essentiel

Les bases de données sont un élément important dans la conception d'une application, qu'il s'agisse d'une application web ou non.

Dans ce cours, nous avons vu qu'il existait deux types de bases de données : relationnelles (SQL) ou non-relationnelles (NoSQL).

Dans le cadre des bases de données relationnelles, nous avons vu qu'il existait différents types de relations, qui permettent d'interconnecter les différentes tables de données entre elles.

Nous avons parcouru quelques exemples, qui montrent comment on peut manipuler ces structures de données à l'aide de Python. Les opérations CRUD (CREATE, READ, UPDATE et DELETE) ont pu être expliquées.

Enfin, nous avons introduit la notion d'ORM, qui fera l'objet d'un prochain cours, et qui vous permettra d'avoir un niveau d'abstraction plus élevé sur la manipulation des bases de données relationnelles.

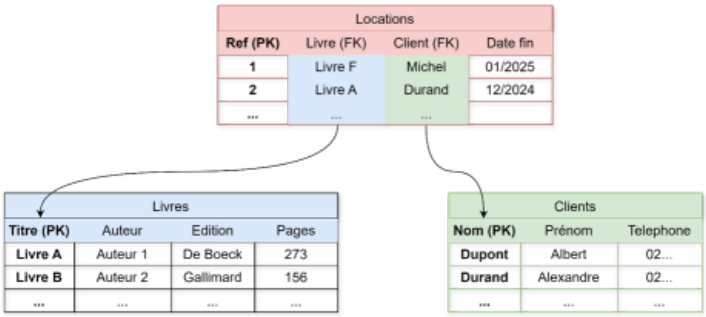
V. Auto-évaluation

A. Exercice

Question 1

[solution n°3 p.15]

Créez une nouvelle base de données, et construisez la base de données de notre bibliothèque. Vous pouvez vous baser sur le diagramme suivant, que nous avons vu ensemble précédemment :



Question 2

[solution n°4 p.16]

Nous souhaitons contacter par téléphone la personne qui a loué le livre 'LivreA'. Ecrivez un code qui permet d'obtenir cette information.

Tous droits réservés à STUDI - Reproduction interdite

B. Test

Exercice 1 : Quiz

[solution n°5 p.16]

Question 1

Les bases de données SQL sont toujours plus performantes que les bases de données NoSQL.

- ○ Vrai
- ○ Faux

Question 2

L'acronyme « *FK* » signifie :

- ○ First Key
- ○ Foreign Key
- ○ Fast key

Question 3

MongoDB est une base de données non-relationnelle.

- ○ Vrai
- ○ Faux

Question 4

Quel est l'intérêt des liens de type 'Foreign Key' ?

- ☐ Il n'y en a pas vraiment, c'est une question de vocabulaire
- ☐ Elles permettent de lier des tables ensemble, et donc de récupérer des données d'une table en fonction du contenu d'une autre
- ☐ Elles permettent d'avoir une meilleure organisation de notre base de données, en disposant de tables 'élémentaires' qui sont ensuite liées dans une table plus complexe

Question 5

Qu'est ce qu'un objet 'Cursor' ?

- ○ Une base de données spécifique
- ○ Un objet qui permet de réaliser des requêtes vers une base de données
- ○ Un type de lien entre bases de données

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 6 Solution n°1

Question 1

Une base de données, c'est…

- ○ Un programme qui gère le stockage des données
- ○ Une solution complexe qu'on met en œuvre uniquement pour des gros projets
- ◉ Un moyen simple et efficace de stocker des données

Une base de données est une solution simple, efficace et performante pour stocker des données, qu'elles soient nombreuses ou non.

Question 2

L'acronyme NoSQL signifie qu'une base de données non-relationnelle ne peut jamais être manipulée à l'aide de requêtes SQL.

- ○ Vrai
- ◉ Faux

C'est faux, No-SQL signifie « *Not-Only SQL* ». Dans certains cas, en fonction du type de données stockée, il est parfois possible d'utiliser un langage SQL pour interagir avec une base de données No-SQL.

Question 3

Une base de données relationnelle peut utiliser des fichiers ou des graphes comme enregistrements.

- ○ Vrai
- ◉ Faux

Dans ce cas, il s'agirait plutôt d'une base de données non-relationnelle, qui manipule des fichiers (JSON par exemple), ou encore des graphes logiques.

Question 4

Qu'est ce qu'une clé primaire ?

- ○ Un enregistrement dans une table
- ◉ Un identifiant unique dans une table
- ○ Un type de relation dans une base de données non-relationnelle

Une clé primaire (« *Primary Key* ») est un identifiant unique qui permet de pointer un enregistrement spécifique dans une table donnée.

Question 5

Combien de types de relations existe-t-il dans les bases de données relationnelles ?

- ○ 1
- ○ 2
- ◉ 3

Tous droits réservés à STUDI - Reproduction interdite

Il en existe 3 types : one-to-one, one-to-many et many-to-many.

Exercice p. 10 **Solution n°2**

Question 1

CRUD est l'acronyme qui désigne …

- (●) Un ensemble d'opérations utiles pour manipuler une base de données
- ( ) Un type de base de données
- ( ) Un ensemble de relations qui peuvent caractériser une base de données

CRUD est l'acronyme des actions CREATE, READ, UPDATE et DELETE. Il s'agit donc d'un ensemble d'opérations qui permettent de manipuler le contenu d'une base de données.

Question 2

L'interaction avec une base de données fonctionne sur base d'un principe requête-réponse.

- (●) Vrai
- ( ) Faux

C'est effectivement le principe d'interaction avec une base de données.

Question 3

SQLite est une base de données relationnelle.

- (●) Vrai
- ( ) Faux

Oui, il s'agit d'une base de données SQL, et donc relationnelle. C'est celle que nous avons utilisé dans les différents exemples abordés.

Question 4

Que ne faut-il pas oublier lorsqu'on souhaite effectuer une modification dans une base de données ?

- (●) Il faut commiter les modifications à l'aide de la fonction 'commit()'
- ( ) Il faut enregistrer son script Python
- ( ) Il faut clôturer la connexion à la base de données

Lorsqu'on souhaite modifier le contenu, il faut appeler la fonction 'commit()' pour confirmer les modifications à effectuer.

Question 5

ORM est l'acronyme de … :

- ( ) One Relation Mapping
- (●) Object Relational Mapper
- ( ) Object Regression Mapper

Tous droits réservés à STUDI - Reproduction interdite

ORM signifie « *Object Relational Mapper* ». Il s'agit d'un moyen de lier des objets à une base de données SQL.

p. 11 **Solution n°3**

Nous allons commencer par créer les tables 'livres' et 'clients', car ces tables ne contiennent pas de « *Foreign keys* », ce sont des tables de base :

```
import sqlite3

myDB = sqlite3.connect("database.db")

cur = myDB.cursor() # Notre objet Cursor

cur.execute("CREATE TABLE livres(titre, auteur, edition, pages)")

cur.execute("CREATE TABLE clients(nom, prenom, telephone)")
```

Une fois les tables élémentaires créées, on peut les remplir :

```
cur.execute("""
    INSERT INTO livres VALUES
        ('LivreA', 'Auteur 1', 'De Boeck', 273),
        ('LivreB', 'Auteur 2', 'Gallimard', 156)
    """)

cur.execute("""
    INSERT INTO clients VALUES
        ('Dupont', 'Albert', 02123456789),
        ('Durand', 'Alexandre', 02123456798)
    """)

myDB.commit()
```

Attention à ne pas oublier de commiter nos modifications.

Créons ensuite notre tables « *locations* » :

```
cur.execute("""CREATE TABLE locations(ref,
            livre,
            client,
            date_fin,
            FOREIGN KEY(livre) REFERENCES livres(titre),
            FOREIGN KEY(client) REFERENCES clients(nom))""")

myDB.commit()
```

Comme on peut le voir, nous avons spécifié que les colonnes 'livre' et 'client' sont liées (foreign keys) aux colonnes 'titre' et 'nom' des tables 'livres' et 'clients'.

Ecrivons deux enregistrements dans cette table :

```
cur.execute("""
    INSERT INTO locations ("ref", "livre", "client", "date_fin") VALUES ('1', 'LivreF',
'Michel', '01/2025')""")

cur.execute("""
    INSERT INTO locations ("ref", "livre", "client", "date_fin") VALUES ('1', 'LivreA',
'Durand', '12/2024')""")
```

Notre table de locations est maintenant opérationnelle.

Tous droits réservés à STUDI - Reproduction interdite

p. 11 Solution n°4

Grâce aux liens établis entre nos bases de données, ceci est extrêmement simple :

```
1 res = cur.execute("SELECT telephone FROM locations, clients WHERE
  locations.client=clients.nom AND locations.livre='LivreA'")
2 print(res.fetchall())
```

Ce qui devrait vous donner le résultat :

```
> [(2123456798)]
```

Ce qui est bien le numéro de Monsieur Durand, qui est le client qui possède le livre 'livreA'.

Exercice p. 12 Solution n°5

Question 1

Les bases de données SQL sont toujours plus performantes que les bases de données NoSQL.

- ○ Vrai
- ◉ Faux

Il n'y a pas de raison de penser que ce soit le cas. En général, cela dépend du nombre de données et du type de données. Le choix se portera plutôt sur les aspects de scalabilité et d'évolution, ou tout simplement sur des contraintes de mise en œuvre.

Question 2

L'acronyme « *FK* » signifie :

- ○ First Key
- ◉ Foreign Key
- ○ Fast key

« *FK* » signifie « *Foreign Key* », qui signifie « *Clé étrangère* ».

Question 3

MongoDB est une base de données non-relationnelle.

- ◉ Vrai
- ○ Faux

Vrai. MangoDB est une base de données de type NoSQL très réputée.

Question 4

Quel est l'intérêt des liens de type 'Foreign Key' ?

- ☐ Il n'y en a pas vraiment, c'est une question de vocabulaire
- ☑ Elles permettent de lier des tables ensemble, et donc de récupérer des données d'une table en fonction du contenu d'une autre
- ☑ Elles permettent d'avoir une meilleure organisation de notre base de données, en disposant de tables 'élémentaires' qui sont ensuite liées dans une table plus complexe

Tous droits réservés à STUDI - Reproduction interdite

Les deux dernières propositions sont correctes. En effet, établir des liens permet d'abord d'avoir une organisation plus simple, en ayant recours à des tables élémentaires génériques. Ensuite, ces liens permettent d'obtenir les informations des tables 'élémentaires' à partir de la table qui utilise des 'Foreign Keys'.

Question 5

Qu'est ce qu'un objet 'Cursor' ?

- ○ Une base de données spécifique
- ◉ Un objet qui permet de réaliser des requêtes vers une base de données
- ○ Un type de lien entre bases de données

Un object 'Cursor' permet d'établir des requêtes qui seront envoyées à notre base de données.

Tous droits réservés à STUDI - Reproduction interdite